

GENERATIVE ADVERSARIAL NETWORKS

SOMMAIRE

1- Modèle

1

- 1.1- Architecture
- 1.2- Relation à la théorie des jeux
- 1.3- Calcul des paramètres des réseaux
- 1.4- Différents modèles

2- Partie pratique

4

Les réseaux antagonistes générateurs (Generative Adversarial Networks, ou GAN) sont des réseaux de neurones apprenant à générer des données de synthèse similaires à des données d'entrées qui sont connues. Ils ont été introduits en 2014 [2] et ne cessent depuis de se développer, avec de nombreuses applications (pour n'en citer que quelques unes : super-résolution [4], génération de vidéos [6], transfert de domaines [7]), et extensions (génération d'images à partir d'un texte [8] par exemple). Ils font partie des modèles dits génératifs, au même titre par exemple que les autoencodeurs variationnels [3].

L'objectif ici est de construire un GAN capable de générer des images simples. À partir de ce modèle, et d'un état de l'art, il sera possible de progresser vers des tâches plus complexes, comme par exemple la génération d'images basse et haute résolution à partir d'une description textuelle (StackGAN, [8], figure 0-1).

1- MODÈLE

1.1- Architecture

Un GAN se compose de deux modèles (figure 1-2), l'un génératif (G), l'autre discriminant (D) (figure 1-2).

G produit des données à partir de variables générées aléatoirement (variables latentes), via un réseau de neurones. D est un classifieur qui détermine si son entrée ressemble à une "vraie" donnée de la base d'entraînement, ou s'il s'agit d'une donnée de synthèse provenant de G. Sous sa forme basique, il s'agit d'un simple réseau convolutif ou d'un perceptron multicouche.

L'entraînement de G vise à accroître le taux d'erreur de D (i.e. faire croire à D que les données générées sont des vraies). La rétropropagation est effectuée sur la négation de la fonction de perte (classification binaire) de D, de sorte qu'à convergence, les données réelles et générées ne puissent être distinguées par D.

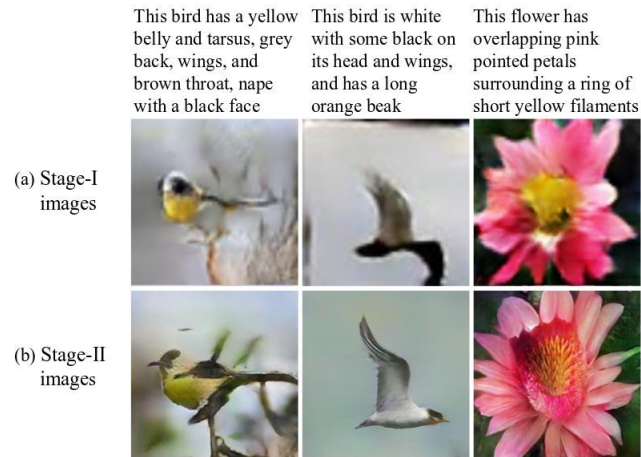


Figure 1. Photo-realistic images generated by our StackGAN from unseen text descriptions. Descriptions for birds and flowers are from CUB [32] and Oxford-102 [18] datasets, respectively. (a) Given text descriptions, Stage-I of StackGAN sketches rough shapes and basic colors of objects, yielding low resolution images. (b) Stage-II of StackGAN takes Stage-I results and text descriptions as inputs, and generates high resolution images with photo-realistic details.

FIGURE 0-1 – Exemple de réalisation de StackGAN (source : [8])

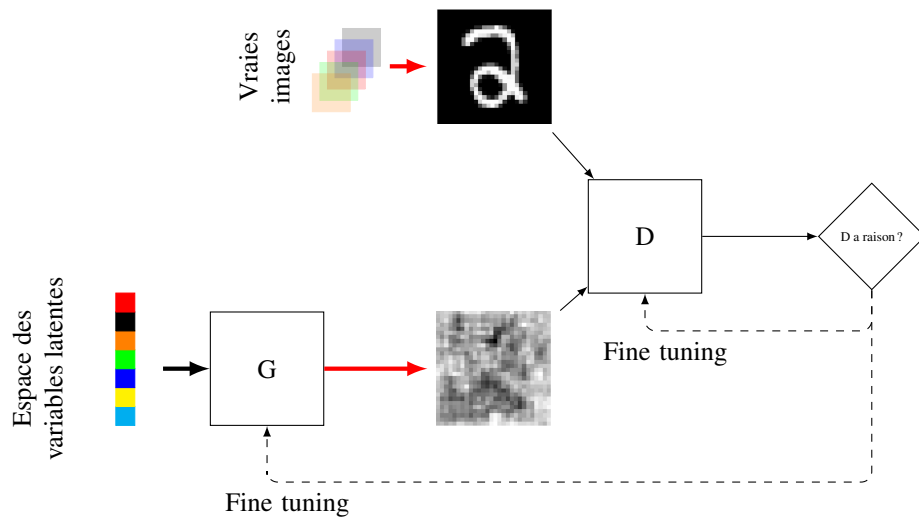


FIGURE 1-2 – Architecture d'un GAN

L'entraînement de D se fait sur un ensemble particulier, composé de vraies données et de données générées par G . D essaye par rétropropagation d'optimiser son taux de bonne classification.

1.2- Relation à la théorie des jeux

On peut aborder les GAN d'un point de vue théorie des jeux : G et D sont deux joueurs, jouant alternativement à un jeu à somme nulle. G transforme un vecteur de bruit $\mathbf{h}$ en un échantillon $\mathbf{x}$. Le réseau D est entraîné de manière binaire, en considérant les échantillons du générateur comme exemples négatifs (tirés

d'une distribution $\mathbf{p}_G$ inconnue) et les vraies données comme exemples positifs (tirés selon $\mathbf{p}_{data}$ inconnue). Comme dit précédemment, G essaye de tromper D dans ce jeu. Une fonction $v(\theta_D, \theta_G)$ vient quantifier la récompense de D , où θ_D et θ_G sont les paramètres de D et G (les poids et biais lorsque les joueurs sont des réseaux de neurones). Le générateur G reçoit $-v(\theta_D, \theta_G)$, puisque le jeu est à somme nulle. Durant le jeu, les deux joueurs essayent de maximiser leur récompense, et l'ensemble se modélise donc comme un problème d'optimisation minmax :

$$(\hat{\theta}_D, \hat{\theta}_G) = \underset{g}{\operatorname{Arg\,min}} \max_d v(d, g)$$

Dans sa forme la plus simple [2] :

$$v(\theta_D, \theta_G) = \mathbb{E}_{\mathbf{x} \sim \mathbf{p}_{data}} (\log[D(\mathbf{x})]) + \mathbb{E}_{\mathbf{h} \sim \mathbf{p}_G} (\log[1 - D(G(\mathbf{h}))])$$

Le premier terme force D à étiqueter des données réelles comme telles (1), le second force D à labeliser les données générées par G comme telles (0). G tente de tromper D en lui faisant étiqueter ses échantillons comme réels (1), essayant donc de minimiser la fonction, tandis que D essaye de la maximiser. Des étapes de descente de gradient permettent d'optimiser cette fonction différentiable, pour éventuellement atteindre un équilibre de Nash. Pour G fixé, le D optimal est donné par :

$$\hat{D}(\mathbf{x}) = \frac{\mathbf{p}_{data}(\mathbf{x})}{\mathbf{p}_{data}(\mathbf{x}) + \mathbf{p}_G(\mathbf{x})}$$

1.3- Calcul des paramètres des réseaux

L'algorithme 1 permet de calculer les paramètres (poids et biais) des réseaux D et G , respectivement regroupés dans des vecteurs θ_D et θ_G .

Algorithme 1 : Calcul des paramètres θ_D et θ_G de D et G dans un GAN

Données : la taille des minibatches m , $\mathbf{p}_G(\mathbf{h})$ et $\mathbf{p}_{data}(\mathbf{x})$

Résultat : les paramètres θ_D et θ_G des réseaux G et D

début

pour nombre_iterations **faire**

pour k étapes **faire**

 Tirer un minibatch de m échantillons $\mathbf{h}^{(i)}$ selon $\mathbf{p}_G(\mathbf{h})$

 Tirer un minibatch de m échantillons $\mathbf{x}^{(i)}$ selon $\mathbf{p}_{data}(\mathbf{x})$

 Mise à jour de D par descente de gradient :

$$\nabla_{\theta_D} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log (1 - D(G(\mathbf{h}^{(i)})) \right]$$

 Tirer un minibatch de m échantillons $\mathbf{h}^{(i)}$ selon $\mathbf{p}_G(\mathbf{h})$

 Mise à jour de G par descente de gradient :

$$\nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^m \left[\log (1 - D(G(\mathbf{h}^{(i)})) \right]$$

L'apprentissage des GAN peut être difficile en pratique lorsque G et D sont des réseaux de neurones (et ce d'autant plus qu'ils sont profonds), et lorsque $\max_d v(d, g)$ est non convexe. En général, G apprend mal sur les premières itérations et D rejette les exemples générés avec confiance. Dans cette situation $\log[1 - D(G(\mathbf{h}))]$ sature. Il est alors possible d'entraîner G pour maximiser $\log[D(G(\mathbf{h}))]$ ce qui assure un meilleur entraînement de G , mais pose d'autres problèmes.

D'autres fonctions objectif existent dans la littérature pour rechercher l'équilibre dans le jeu.

1.4- Différents modèles

Les GAN ont été introduits en 2014 ([2]), et ont donné lieu à de nombreuses déclinaisons (figure 1-3) à partir du modèle original ou utilisant d'autres types de réseaux..

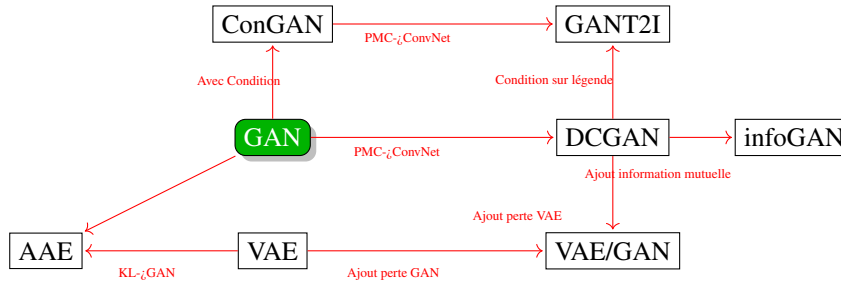


FIGURE 1-3 – Quelques déclinaisons des GAN

Nous citons ici quelques exemples parmi les nombreuses variations existantes¹ :

- **ConGAN** (Conditional GAN) [5] : G et D ont pour entrée supplémentaire le résultat attendu (par exemple la classe dans le cadre des données MNIST), de sorte que les réseaux produisent $D(x | y)$ et $G(x | y)$
- **InfoGAN** [1] propose de décomposer $\mathbf{h} = [\mathbf{z} \mathbf{c}]^\top$, où $\mathbf{z}$ est un bruit incompressible et $\mathbf{c}$ est le code latent, dont l'objectif est de cibler la sémantique des descripteurs de $\mathbf{p}_{data}$. L'optimisation proposée est alors

$$\min_g \max_d (v(d, g) - \lambda I(\mathbf{c}, G([\mathbf{z} \mathbf{c}]))$$

avec I l'information mutuelle. Maximiser le second terme impose à $\mathbf{c}$ de contenir autant que possible des informations importantes sur les vrais exemples.

- **cGANs** [5], ou GAN conditionnels, dans lesquels G et D sont conditionnés à une information supplémentaire $\mathbf{y}$. La fonction objectif est alors

$$\min_g \max_d \mathbb{E}_{\mathbf{x} \sim \mathbf{p}_{data}} (\log[D(\mathbf{x}|\mathbf{y})]) + \mathbb{E}_{\mathbf{h} \sim p_{\mathbf{h}}} (\log[1 - D(G(\mathbf{h}|\mathbf{y}))])$$

L'information $\mathbf{y}$ peut être la classe d'un objet, du texte, des points d'intérêt par exemple.

- **CycleGAN** [9] dont l'objectif est d'apprendre une correspondance entre deux données non appariées et issues de deux domaines différents (adaptation de domaine). Si le cas apparié a été traité assez tôt (on dispose de deux observations d'un même phénomène avec deux capteurs différents, et l'on souhaite apprendre la correspondance entre ces deux capteurs), le cas non apparié (pas d'observation simultanée) est plus délicat.

2- PARTIE PRATIQUE

Proposer à l'aide du TP sur les réseaux convolutifs un GAN utilisant de tels réseaux comme générateur et discriminateur. L'application est la génération de chiffres manuscrits, à partir d'un apprentissage réalisé sur la base MNIST. A partir du notebook fourni, votre travail consiste à :

- déclarer les réseaux convolutifs G et D adaptés (figure 2-5).
- calculer les fonctions de perte

$$lossG = -\frac{1}{m} \sum_{i=1}^m \log(D[G(z)])$$

et

$$lossD = -\frac{1}{m} \sum_{i=1}^m \log(1 - D[G(z)]) + \log(D(x))$$

1. Le site <https://github.com/hindupuravinash/the-gan-zoo> maintient une liste des déclinaisons des GANs

où m est la taille du batch, z le vecteur des variables latentes, x l'image d'entraînement (vraie image) proposée au réseau D .

Le GAN produit des images telles que celles illustrées sur la figure 2-4.



FIGURE 2-4 – Génération de quelques images à partir d'un apprentissage d'un GAN sur MNIST (50000 epochs)

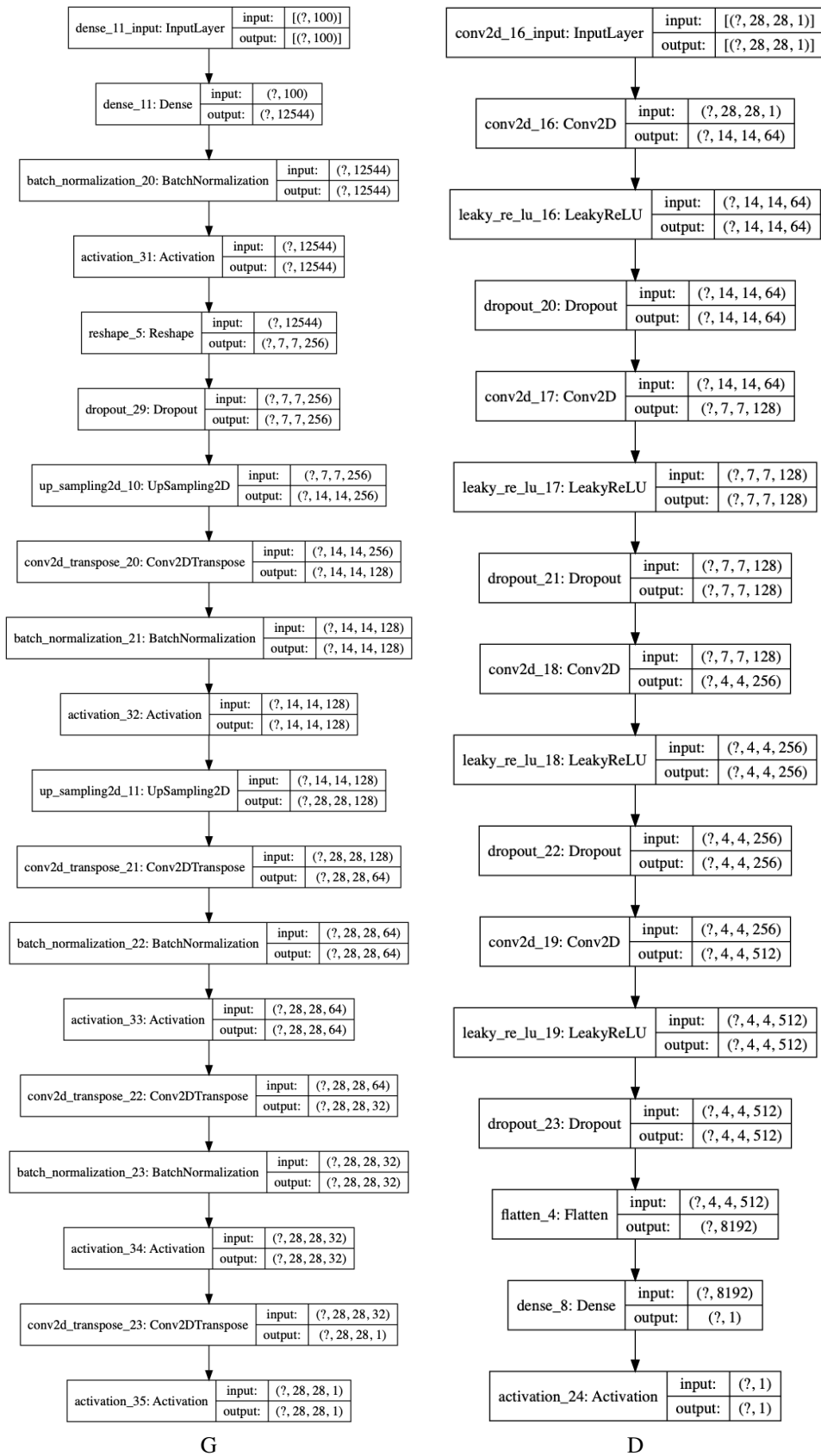


FIGURE 2-5 – Réseaux convolutifs G et D)

- [1] Xi Chen, Yan Duan, Rein Houthoofd, John Schulman, Ilya Sutskever, and Pieter Abbeel. Infogan : Interpretable representation learning by information maximizing generative adversarial nets. CoRR, abs/1606.03657, 2016.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, Advances in Neural Information Processing Systems 27, pages 2672–2680. Curran Associates, Inc., 2014.
- [3] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. CoRR, abs/1312.6114, 2013.
- [4] Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew P. Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, and Wenzhe Shi. Photo-realistic single image super-resolution using a generative adversarial network. CoRR, abs/1609.04802, 2016.
- [5] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. CoRR, abs/1411.1784, 2014.
- [6] Carl Vondrick, Hamed Pirsiavash, and Antonio Torralba. Generating videos with scene dynamics. CoRR, abs/1609.02612, 2016.
- [7] Donggeun Yoo, Namil Kim, Sunggyun Park, Anthony S. Paek, and In-So Kweon. Pixel-level domain transfer. CoRR, abs/1603.07442, 2016.
- [8] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaolei Huang, Xiaogang Wang, and Dimitris N. Metaxas. Stackgan : Text to photo-realistic image synthesis with stacked generative adversarial networks. CoRR, abs/1612.03242, 2016.
- [9] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017, pages 2242–2251. IEEE Computer Society, 2017.